

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: Симуляция роевого интеллекта на GPU

Студент гр. 4344

Х. Нгуен

Руководитель

Р.П. Шестопалов

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Х. Нгуен

Группа: 4344

Тема практики: Симуляция роевого интеллекта на GPU

Задание на практику:

Разработка трехмерной симуляции поведения стаи на основе алгоритма Крейга Рейнольдса с использованием графического API WebGPU (Dawn), вычислительных шейдеров на WGSL и языка программирования C++. Реализация интерактивного пользовательского интерфейса для динамической настройки параметров симуляции.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 18.07.2026

Дата защиты отчета: 18.07.2026

Студент

Х. Нгуен

Руководитель

Р.П. Шестопалов

АННОТАЦИЯ

Данный отчет посвящен разработке трехмерной симуляции поведения стаи, основанной на алгоритме Крейга Рейнольдса. В работе рассматривается применение современного графического API WebGPU (Dawn) и языка программирования C++ для создания интерактивного приложения. Особое внимание уделено использованию вычислительных шейдеров для распараллеливания физических вычислений на стороне графического процессора, что позволило программе поддерживать большее количество агентов в симуляции. Также описан процесс реализации пользовательского интерфейса для динамического управления параметрами среды.

SUMMARY

This report is dedicated to the development of a high-performance 3D simulation of flocking behavior based on Craig Reynolds' Boids algorithm. The paper explores the application of the modern graphics API WebGPU (Dawn) and the C++ programming language to create an interactive application. Special attention is given to the use of compute shaders for parallelizing physical calculations on the GPU, which allowed the program to support a larger number of agents in the simulation. The process of integrating a user interface for dynamic environment parameter control is also described.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область.....	6
1.2 Задачи практики	8
1.3 Математическая модель симуляции коллективного поведения	10
2 РЕЗУЛЬТАТЫ РАБОТЫ	13
2.1. Подготовка базового рендера	13
2.2. Проектирование математической модели и структур данных ..	14
2.3. Реализация вычислительного конвейера на GPU	15
2.4. Интеграция базового графического интерфейса пользователя. ..	18
2.5. Отладка и оптимизация.....	20
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	21
4 ОТЗЫВ РУКОВОДИТЕЛЯ	23
ЗАКЛЮЧЕНИЕ	23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	25

ВВЕДЕНИЕ

В современном мире компьютерной графики и симуляции физических процессов одной из наиболее актуальных задач является обеспечение высокой производительности при моделировании систем с большим числом интерактивных объектов. Мультиагентные системы, такие как симуляции коллективного поведения (стаи, роя, толпы), требуют значительных вычислительных ресурсов для расчета межагентных взаимодействий, сложность которых растет квадратично с увеличением количества элементов. С развитием современных графических API нового поколения, таких как WebGPU, Vulkan, DirectX12 и распространением технологии вычислительных шейдеров появилась возможность перенести вычисления на графический процессор. Это позволяет распараллелить физические расчеты на тысячи потоков, значительно снизить нагрузку на центральный процессор и обрабатывать сотни тысяч независимых агентов в реальном времени при стабильной частоте кадров.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Предметной областью работы является проектирование и разработка систем трехмерной визуализации и интерактивного моделирования физических процессов реального времени. В качестве базовой модели коллективного поведения в проекте рассматривается классический алгоритм стаи Крейга Рейнольдса, описывающий скоординированное движение множества независимых агентов на основе локальных правил взаимодействия [1].

Основной техникой вызов при реализации подобных мультиагентных систем заключается в высокой вычислительной сложности алгоритма, которая при поиске соседних объектов возрастает квадратично. Для обеспечения плавной визуализации больших графических массивов в данной работе применяется перенос физических расчетов на сторону графического процессора.

Разрабатываемое приложение совмещает в себе подходы параллельного программирования на видеокартах и построение интерактивных интерфейсов, что позволяет динамически управлять физическими параметрами симуляции непосредственно в процессе рендеринга.

Результаты выполненной работы и разработанные программные решения могут быть применены в следующих областях:

- **Игровая индустрия и компьютерная графика:** моделирование поведения больших скоплений живых организмов (косяков рыб, роев насекомых, стай птиц, толпы людей), а также оптимизация систем частиц.
- **Робототехника и беспилотные системы:** разработка алгоритмов децентрализованного группового управления флотилиями автономных дронов или роботов, функционирующих без единого координационного центра.

- **Научные исследования:** моделирование динамических систем, процессов самоорганизации в живой и неживой природе, а также изучение подходов к распараллеливанию сложных математических расчетов.

1.2 Задачи практики

В соответствии с утвержденным планом разработки, для достижения поставленной цели в рамках учебной практики были сформулированы и решены следующие задачи:

1. Подготовка базового рендера:

- Настройка базового графического конвейера.
- Вывод тестовой геометрической фигуры на экран для проверки корректности инициализации графического устройства, кадрового буфера и работы базовых вершинных и фрагментных шейдеров.

2. Проектирование математической модели:

- Изучение теоретических основ и математического аппарата алгоритма Крейга Рейнольдса для симуляции коллективного поведения стаи.
- Определение и структурирование перечня параметров среды и агентов, которые необходимо хранить в памяти и передавать на графический процессор для корректного расчета физики движения.

3. Реализация симуляции на GPU с помощью вычислительного шейдера:

- Написание вычислительного шейдера для реализации и расчета векторов сил Рейнольдса.
- Проектирование и оптимизация схемы отправки данных между центральным процессором и графическим процессором с учетом требований к выравниванию памяти.
- Организация графического конвейера для отрисовки трехмерных частиц на основе координатных и скоростных данных, рассчитанных непосредственно в вычислительном шейдере.

4. Интеграция базового графического интерфейса:

- Создание интерактивного окна управления пользовательского интерфейса для изменения физических параметров и коэффициентов сил стаи непосредственно во время рендеринга.

5. Отладка, полировка и оптимизация:

- Проведение тестирования производительности графического конвейера и вычислительного ядра при масштабировании системы.
- Тестирование корректности поведения роя и стабильности работы графических окон при динамическом изменении параметров через интерфейс.
- Проведение финального рефакторинга, подготовка итоговой кодовой базы проекта и написание отчета о проделанной учебной работе.

1.3 Математическая модель симуляции коллективного поведения

Для математического описания поведения стаи используется расширенная трехмерная модель Крейга Рейнольдса [2]. Движение каждого независимого *boi*d-агента в пространстве определяется его вектором положения $\vec{P}$ и вектором скорости $\vec{V}$.

1.3.1 Локальные правила взаимодействия агентов

Вектор результирующего ускорения каждого агента формируется на основе вычисления локальных сил взаимодействия с соседними агентами, находящимися в пределах его радиуса видимости R_{visual} и угла обзора θ_{vision} [3]. В разработанной модифицированной модели рассчитываются следующие составляющие:

- **Сила сплоченности (Cohesion):** направляет агента к локальному центру масс соседних дружественных агентов.
- **Сила выравнивания (Alignment):** сонаправляет вектор скорости агента со средним вектором скорости его соседей.
- **Сила разделения (Separation):** предотвращает столкновение агентов, генерируя силу отталкивания, обратно пропорциональную расстоянию до соседа, если оно меньше безопасного радиуса $R_{protected}$.
- **Сила отторжения чужаков (Strange Force):** действует между агентами разных фракций (стай) при включенном режиме разделения. Данная сила имеет увеличенный радиус действия $R_{stranger}$ и повышенный коэффициент отталкивания для предотвращения смешивания стай.
- **Сила избегания границ (Boundary Avoidance):** обеспечивает удержание агентов внутри виртуального куба размером L_{cube} . При приближении к границе куба на расстояние меньше порогового M_{margin} к вектору скорости плавно добавляется сила разворота, направленная внутрь расчетной области.

Схематичное представление основных векторов сил, действующих на отдельного *boi*d-агента со стороны соседей и физических границ (см. рис. 1).

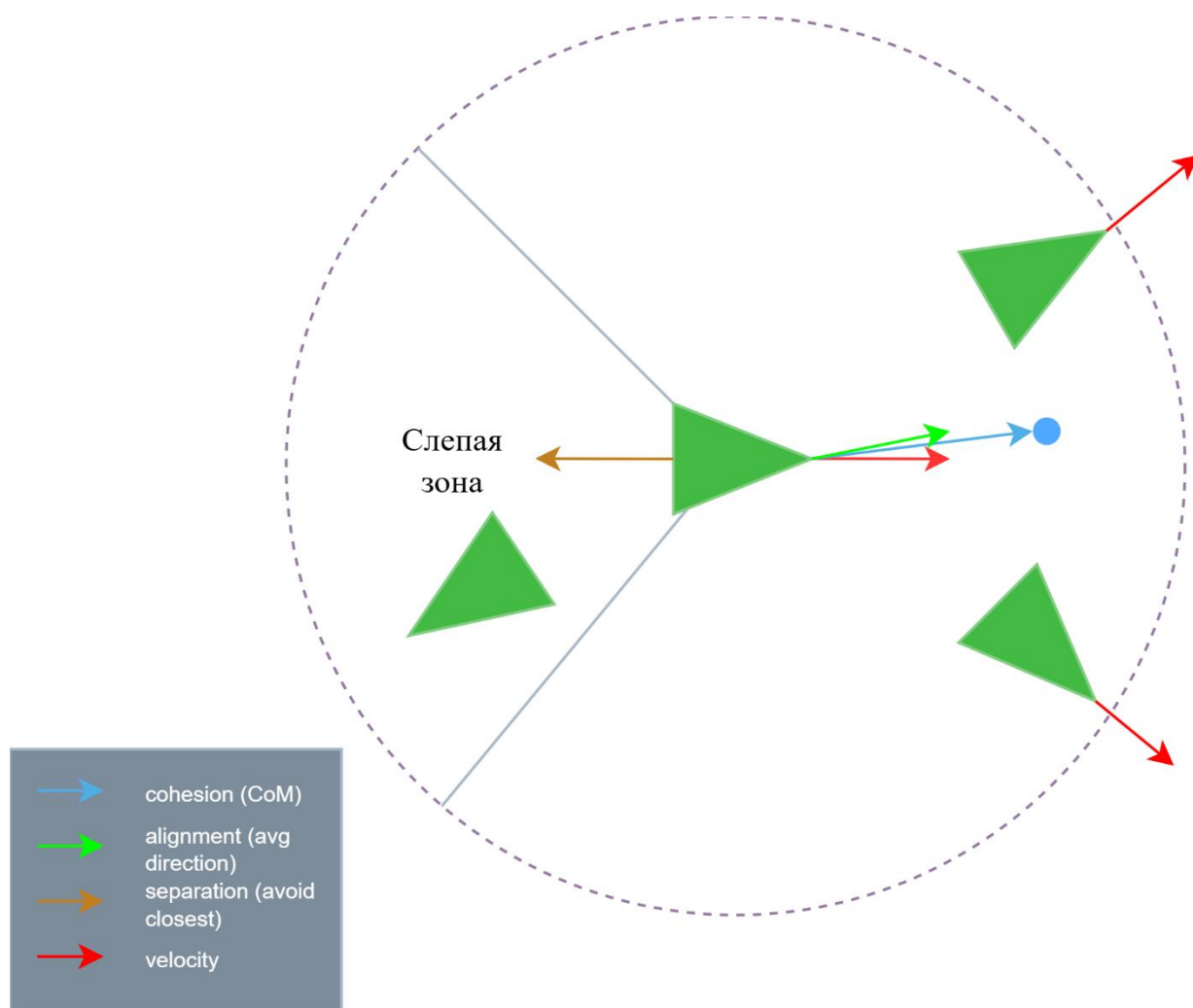


Рисунок 1 - Схема основных векторов сил и областей видимости boïd-агента

Для обеспечения корректного расчета физики движения на GPU через Uniform-буфер передается структура параметров симуляции. Перечень настраиваемых параметров, их физический смысл и базовые значения приведены в таблице 1.

Таблица 1 - Физические параметры среды и boïd-агентов

Показатель	Единица измерения	Значение по умолчанию	Физический смысл
visualRange	ед. пространства	2.0	Радиус кругового агента
protectedRange	ед. пространства	0.5	Радиус зоны безопасности разделения
strangerProtectedRange	ед. пространства	2.5	Расстояние реакции на агентов чужой стаи

Продолжение таблицы 1

maxSpeed	ед. скорости / сек	5.0	Максимальный предел скорости полета
minSpeed	ед. скорости / сек	2.0	Минимальный предел скорости полета
cubeSize	ед. пространства	4.5	Половина длины ребра ограничивающего куба
cohesionFactor	коэф. влияния	0.005	Множитель силы притяжения к центру масс
alignmentFactor	коэф. влияния	0.05	Множитель силы сонаправления скоростей
separationFactor	коэф. влияния	0.05	Множитель базовой силы отталкивания от соседей
turnFactor	коэф. влияния	0.15	Сила разворота агента при приближении к границе
strangeForceFactor	коэф. влияния	10.0	Коэффициент жесткости отталкивания чужих стай
visionRadius	градусы	240.0	Угол обзора (слепая зона находится позади вектора скорости)
margin	ед. пространства	1.0	Ширина буферной зоны куба для начала разворота
activeBoidsCount	ед.	1000	Количество одновременно симулируемых агентов
divideFlocks	логический флаг	0	Режим разделения на независимые цветовые фракции

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Подготовка базового рендера

На начальном этапе работы был настроен базовый графический конвейер для проверки корректности инициализации графического API WebGPU.

В среде разработки на языке C++ был реализован класс инициализации графического контекста, выполняющий следующие последовательные операции:

1. Инициализация экземпляра `wgpu::Instance`.
2. Запрос физического графического адаптера с поддержкой современных низкоуровневых API (Vulkan, Direct3D 12 или Metal в зависимости от целевой операционной системы).
3. Создание логического устройства и очереди команд.
4. Настройка цепочки создания изображений для обеспечения двойной буферизации кадра.

Для тестирования корректности работы конвейера были написаны простейшие вершинный и фрагментный шейдеры на языке WGSL, осуществляющие позиционирование и закраску вершин тестовой модели агента. Успешный вывод геометрической фигуры на экран подтвердил стабильность работы инициализированного контекста, отсутствие утечек памяти в графическом цикле и готовность конвейера к отрисовке более сложных динамических объектов.

2.2. Проектирование математической модели и структур данных

На основе математического аппарата алгоритма Рейнольдса была спроектирована структура данных для хранения физического состояния системы. Для обеспечения максимальной производительности при передаче данных между центральным процессором и графическим процессором используется выравнивание памяти по границам 16 байт.

Для управления симуляцией на стороне GPU была спроектирована структура параметров `SimulationParams`, передаваемая через Uniform-буфер:

- Дистанционные пороги видимости (`visualRange`, `protectedRange`, `strangerProtectedRange`).
- Ограничители скоростей (`maxSpeed`, `minSpeed`).
- Геометрические параметры пространства (`cubeSize`, `margin`).
- Физические коэффициенты сил (`cohesionFactor`, `alignmentFactor`, `separationFactor`, `turnFactor`, `strangeForceFactor`).
- Угол обзора агента (`visionRadius`), определяющий «слепую зону».
- Системные переменные (`activeBoidsCount` — число активных агентов, `divideFlocks` — флаг разделения на стаи).

Для описания состояния каждого отдельного агента была разработана структура `BoidData`:

```
struct BoidData {  
    position: vec4<f32>,  
    velocity: vec4<f32>,  
    centerOfMass: vec4<f32>,  
};
```

Использование векторов типа `vec4<f32>` вместо `vec3<f32>` обусловлено требованиями аппаратного выравнивания памяти в WebGPU. Четвертая компонента вектора положения используется для хранения уникального идентификатора стаи, что минимизирует накладные расходы на передачу дополнительных буферов.

2.3. Реализация вычислительного конвейера на GPU

Физический расчет межагентных взаимодействий реализован в вычислительном шейдере на языке WGSL. Вычислительный конвейер использует один Uniform-буфер для параметров среды и два буфера хранения для данных агентов.

Для исключения состояния гонки данных, при котором потоки GPU считывают координаты агентов, уже обновленные другими потоками в рамках текущего кадра, была применена техника двойной буферизации на GPU:

- `@binding(1) var<storage, read> boidsIn` — буфер, содержащий координаты и скорости агентов с предыдущего шага симуляции (доступен только для чтения).
- `@binding(2) var<storage, read_write> boidsOut` — буфер для записи обновленных физических параметров текущего шага.

После каждого расчетного кадра указатели буферов `boidsIn` и `boidsOut` меняются местами.

Вычислительный шейдер оптимизирован для параллельного исполнения. Размер рабочей группы установлен равным 64 потокам, что обеспечивает оптимальную утилизацию вычислительных блоков современных видеокарт.

Алгоритм работы вычислительного шейдера `compute_main` для каждого отдельного потока с индексом `index` состоит из следующих шагов:

1. **Проверка границ:** если глобальный индекс потока превышает количество активных агентов (`params.activeBoidsCount`), выполнение шейдера завершается.
2. **Инициализация аккумуляторов сил:** сбрасываются счетчики соседей, векторы центра масс, средней скорости и силы разделения.
3. **Фильтрация соседей (внутренний цикл):** производится итеративный обход всех активных агентов симуляции. Для каждого соседа выполняются проверки:

- Исключение самодействия ($j == \text{index}$).
- Расчет расстояния dist между текущим агентом и соседом j .
- **Проверка слепой зоны:** с помощью скалярного произведения текущего вектора направления движения агента boidDir и направления на соседа toNeighborDir проверяется попадание соседа в поле зрения: $\text{if } (\text{dot}(\text{boidDir}, \text{toNeighborDir}) > \text{params.visionRadius})$
- **Проверка фракции:** если включен режим divideFlocks , проверяется совпадение идентификаторов стай дружественных агентов.

4. Расчет локальных сил:

- Для дружественных агентов в пределах visualRange накапливаются координаты для центра масс и векторы скоростей для выравнивания. Если дружественный агент находится ближе protectedRange , рассчитывается вектор силы разделения, обратно пропорциональный квадрату расстояния.
- Для чужеродных агентов в пределах увеличенного радиуса $\text{strangerProtectedRange}$ рассчитывается вектор силы отталкивания чужаков, масштабируемый коэффициентом $\text{strangeForceFactor}$.

5. Интегрирование сил Рейнольдса:

- На основе накопленных данных вычисляются результирующие векторы сил сплоченности и выравнивания, которые масштабируются соответствующими факторами влияния.
- Физический центр масс текущего агента плавно интерполируется с предыдущим состоянием во избежание резких колебаний («дергания») геометрии: $\text{let smoothCoM} = \text{mix}(\text{oldCoM}, \text{centerOfMass.xyz}, 0.1);$
- К результирующей скорости прибавляется вектор силы разделения.

6. **Ограничение скорости:** производится проверка длины результирующего вектора скорости. Если скорость превышает `maxSpeed` или оказывается меньше `minSpeed`, вектор нормализуется и масштабируется до граничных значений.
7. **Избегание границ расчетной области:** если агент приближается к границам невидимого куба `cubeSize` на расстояние пороговой зоны `margin`, к соответствующим компонентам вектора скорости добавляется или вычитается разворачивающая сила `turnFactor`.
8. **Обновление состояния:** новое положение агента рассчитывается методом Эйлера:

$$\vec{P}_{new} = \vec{P}_{old} + \vec{V} \cdot dt$$

Результат записывается в выходной буфер `boidsOut`.

2.4. Интеграция базового графического интерфейса пользователя.

Для интерактивного взаимодействия с симуляцией в проект была внедрена библиотека Dear ImGui, адаптированная для работы с контекстом WebGPU и графической библиотекой GLFW [4].

В рамках разработки интерфейса был реализован двухкомпонентный оконный интерфейс:

1. **Окно быстрого переключения:** компактное полупрозрачное оверлей-окно в левом верхнем углу экрана, содержащее единственную кнопку-переключатель для скрытия или вызова основной панели управления. Данное решение является универсальным и освобождает рабочую область экрана для наблюдения за симуляцией.
2. **Панель управления ("Control Panel"):** многофункциональное окно, параметры в котором структурированы по раскрывающимся вкладкам:
 - *Distance:* управление радиусами видимости и безопасности агентов.
 - *Cube Params:* изменение физических размеров ограничивающего куба.
 - *Boids Speed:* настройка динамических порогов скоростей.
 - *Factors:* настройка весов сил Рейнольдса.
 - *Simulation Rules:* ползунок для динамического изменения числа активных boid-агентов, а также флаг разделения на стаи.
 - *Debug Options:* включение отрисовки отладочных векторов скоростей и линий центров масс.
 - *Camera Settings:* переключение режимов работы трехмерной камеры (свободный полет или орбитальное вращение вокруг центра роя).
 - *System & Performance:* кнопка переключения полноэкранного режима, слайдер ограничения частоты кадров и счетчик кадров.

Обновление параметров, измененных пользователем через интерфейс, происходит на лету: на каждом шаге рендеринга C++ приложение производит

перезапись Uniform-буфера params на графическом процессоре, обеспечивая мгновенную реакцию симуляции на действия пользователя.

2.5. Отладка и оптимизация

В ходе финального этапа разработки было проведено комплексное тестирование и оптимизация созданного программного обеспечения.

Особое внимание было уделено стресс-тестированию производительности вычислительного конвейера на GPU при изменении масштаба симуляции. Поскольку теоретическая сложность алгоритма поиска соседей составляет $O(N^2)$, вычисления на CPU приводили к падению частоты кадров ниже комфортных 60 кадров/сек уже при $N = 3000$ агентов. Использование вычислительных шейдеров на GPU позволило распараллелить расчеты и достичь стабильной частоты кадров в реальном времени при значительно больших масштабах.

Несмотря на высокую производительность базового алгоритма на GPU, асимптотическая сложность поиска соседей остается квадратичной. В качестве потенциального вектора дальнейшей оптимизации рассматривалось внедрение алгоритма пространственного разбиения (Uniform Spatial Grid — деление трехмерного пространства на кубические ячейки). Это позволило бы локализовать поиск соседей исключительно в смежных ячейках и значительно снизить вычислительную сложность. Однако реализация пространственной сетки на GPU требует сложной синхронизации памяти и внедрения параллельной сортировки (например, Bitonic Sort). Подобная архитектурная переработка является нетривиальной задачей, выходящей за временные рамки текущей учебной практики, в связи с чем данная оптимизация была оставлена в качестве основного направления для будущих доработок проекта.

Также было протестировано поведение системы при изменениях параметров через графический интерфейс (например, мгновенное переключение флага разделения стай или скачкообразное увеличение числа агентов). Приложение продемонстрировало высокую стабильность: динамическое перераспределение памяти под буферы GPU и автоматический перезапуск цепочки создания изображений при изменении размеров оконного пространства происходят без сбоев видеодрайвера.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В процессе выполнения задач учебной практики и разработки программного комплекса симуляции стали применяться следующий стек технологий, языков программирования и специализированных библиотек:

- **C++ (стандарт C++17):** Основной язык программирования проекта. Использовался для реализации архитектуры приложения, управления памятью, обработки структур данных на стороне хоста (CPU) и взаимодействия с низкоуровневым графическим API.
- **WebGPU (в реализации Google Dawn):** Кроссплатформенный графический программный интерфейс нового поколения, разрабатываемый консорциумом W3C [5]. Выступал в качестве основного инструмента для прямого взаимодействия с графическим ускорителем, управления графическим конвейером и организации высокопроизводительных параллельных вычислений.
- **WGSL (WebGPU Shading Language):** Строго типизированный язык написания шейдеров для WebGPU [6]. Применялся для разработки вычислительных шейдеров (реализация физики межагентных взаимодействий алгоритма Рейнольдса) и графических шейдеров (вершинных и фрагментных этапов для отрисовки трехмерной геометрии).
- **GLFW:** Легковесная кроссплатформенная библиотека (C/C++). Использовалась для создания и управления системным окном приложения, обработки событий ввода-вывода (опросы клавиатуры и мыши) и инициализации графического контекста (Surface) для привязки к WebGPU.
- **GLM (OpenGL Mathematics):** Математическая библиотека, синтаксис которой основан на спецификации шейдерного языка GLSL. Применялась для выполнения пространственных расчетов на стороне CPU: работы с векторами, матрицами трансформации (модели, вида,

проекции) и реализации математического аппарата трехмерной свободной и орбитальной камеры.

- **Dear ImGui:** C++ библиотека для создания графических пользовательских интерфейсов методом непосредственного режима. Использовалась для интеграции интерактивной панели управления (изменение параметров среды, порогов скоростей, коэффициентов сил) и отладочных метрик поверх основного графического окна приложения.
- **tiny_obj_loader:** Легковесная библиотека (header-only) для синтаксического разбора файлов трехмерных полигональных моделей. Применялась для парсинга и загрузки геометрии агентов из файлов формата .obj с последующим формированием вершинных буферов.
- **CMake:** Кроссплатформенная система автоматизации сборки программного обеспечения из исходного кода. Использовалась для генерации проектных файлов, управления деревом исходных кодов и линковки всех вышеперечисленных внешних зависимостей.
- **Blender:** Свободный профессиональный пакет для создания трехмерной компьютерной графики. Использовался для самостоятельного полигонального моделирования геометрической модели агента симуляции (птицы) с последующим экспортом геометрии и нормалей в формат .obj для загрузки в графический конвейер приложения.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Отзыв руководителя с оценкой (см. рис. 2).

Отзыв
о прохождении учебной практики

Нгуен Хоанг Шон, студент группы 4344, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил учебную практику по теме “Симуляция роевого интеллекта на GPU” с 06.07.2026 по 18.07.2026.

В течение практики обучающийся Нгуен Хоанг Шон выполнял задачи по проектированию и разработке симуляции мультиагентной системы на базе алгоритма Крейга Рейнольдса. В рамки задания входила настройка графического конвейера с использованием API WebGPU (Dawn), написание высокопроизводительных вычислительных шейдеров на языке WGSL, а также интеграция интерактивного пользовательского интерфейса.

В результате практики обучающийся успешно разработал кроссплатформенное программное обеспечение, способное за счет вычислительных мощностей GPU обрабатывать поведение сотен тысяч независимых агентов в реальном времени, полностью реализовав заявленный функционал.

По результатам работы Нгуен Хоанг Шона учебную практику оцениваю на «Отлично».

Руководитель практики
Ассистент каф. МОЭВМ
15.07.2026


Р.П. Шестопалов

Рисунок 2 - Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В ходе прохождения учебной практики была успешно достигнута поставленная цель — разработана высокопроизводительная трехмерная симуляция коллективного поведения стаи на основе модифицированного алгоритма Крейга Рейнольдса.

В результате выполнения работы были решены все поставленные задачи:

1. Успешно настроен и запущен графический конвейер рендеринга с использованием современного кроссплатформенного API WebGPU (Dawn) и языка программирования C++.
2. Спроектирована расширенная математическая модель симуляции, учитывающая слепые зоны агентов, разделение на цветовые фракции и обработку физических границ трехмерного пространства.
3. Разработаны вычислительные шейдеры на языке WGSL. Перенос физических расчетов сложностью $O(N^2)$ на графический процессор обеспечил возможность отрисовки до нескольких десятков тысяч независимых агентов в реальном времени, продемонстрировав высокую эффективность GPU-вычислений.
4. Интегрирован графический пользовательский интерфейс, позволяющий осуществлять динамическую настройку физических параметров среды и визуализации без перезапуска приложения.
5. Проведено комплексное профилирование и стресс-тестирование стабильности программного обеспечения, подтвердившее надежность выбранных архитектурных решений и высокую оптимизацию конвейера.

За время прохождения практики были получены навыки работы с современными графическими API, написания шейдеров и проектирования архитектуры приложения. Разработанная программа и её исходный код размещены в открытом репозитории на платформе GitHub по адресу: <https://github.com/con2222/Dawn-Boids-3D>.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Reynolds, C. W. Flocks, herds and schools: A distributed behavioral model / C. W. Reynolds // ACM SIGGRAPH Computer Graphics. – 1987. – Vol. 21, no. 4. – P. 25-34.
2. Reynolds C. Boids: Background and Update [Электронный ресурс]. URL: <http://www.red3d.com/cwr/boids/> (дата обращения: 10.07.2026).
3. Parker C. Boids Pseudocode [Электронный ресурс]. URL: <http://www.vergenet.net/~conrad/boids/pseudocode.html> (дата обращения: 10.07.2026).
4. ImGui Explorer [Электронный ресурс] / pthom. URL: https://pthom.github.io/imgui_explorer/ (дата обращения: 10.07.2026).
5. WebGPU API Specification [Электронный ресурс] // W3C Working Draft. URL: <https://www.w3.org/TR/webgpu/> (дата обращения: 06.07.2026).
6. WebGPU Shading Language (WGSL) [Электронный ресурс] // W3C Working Draft. URL: <https://www.w3.org/TR/WGSL/> (дата обращения: 06.07.2026).